8. Generative AI, Part 2

Prompting, retrieval, and images

Week 8 · CE/ARCE 2XX AI Literacy for Civil and Architectural Engineering



COLLEGE OF ENGINEERING
Civil & Architectural
Engineering & Mechanics

Agenda

Today's plan

1. Quiz on Week 7
2. Prompting as specification
3. Four failure modes to expect
4. Retrieval-augmented generation (RAG)
5. Beyond text: multimodal models and image generation
6. Cost, energy, and evaluating outputs
7. Lab day: Lab 08 · HW4 posted today

Recap

Last week in 60 seconds

1 Next-token prediction

An LLM scores every possible next token; writing = predict, pick, append, repeat.

2 Tokens and embeddings

Text is split into tokens; each becomes a vector; similar meanings get nearby vectors.

3 Attention and training

Tokens share information through attention; pretraining, then instruction and preference tuning.

4 Plausible ≠ true

Temperature sets randomness, not truth. Nothing in generation checks facts — so it hallucinates.

Learning objectives

By the end of today, you can…

- ✓ Write a prompt as a **specification**: role, context, task, constraints, format, examples, assumptions
- ✓ Recognize four **failure modes** — invented citations, unit slips, overconfidence, sycophancy
- ✓ Explain the **RAG** pipeline (chunk → embed → retrieve → generate) and where it fails
- ✓ Explain how **diffusion** models make images, and why a concept image is not a design
- ✓ **Evaluate** LLM outputs against known answers and sources, with cost and energy in mind

01

Prompting as specification

Tell the model what an engineer would need to know

Prompting

A good prompt reads like a specification

Part	What to write	Example
Role	Who it should act as	“You are helping a student check a calculation.”
Context	Facts it can't know	Project, location, code edition, units
Task	One clear verb	Estimate, check, summarize, compare
Constraints	Limits and rules	SI units, cite sources, ≤ 150 words, no guessing
Output format	Something you can check	A table, numbered steps, or JSON
Examples (few-shot)	Show the pattern you want	1–3 input → output pairs
Assumptions + steps	Make errors visible	“List assumptions; show each step with units.”

Prompting

Before and after: the same question, specified

Before

- "How big should a beam be for a 6 m span?"
- Missing: material, load, supports, units
- Gaps get filled with silent guesses
- Result: a confident rule of thumb

→

After

- **Role:** check a student's calculation
- **Context:** steel beam, simply supported, 6 m, 12 kN/m factored
- **Task:** max moment; plastic modulus (Fy = 345 MPa, φ = 0.9)
- **Format:** SI units each step, assumptions, one-line answer

Now every step is checkable: $M = qL^2/8 = 54$ kN·m (q = load); $Z \geq M/(\varphi F_y) \approx 174 \times 10^3$ mm³

In Python, the specification goes into the messages

```
messages = [
  {"role": "system", "content":
     "You help students check hand calculations. "
     "Use SI units. Show formula, numbers, and "
     "units at each step. List your assumptions. "
     "If information is missing, say so."},
  {"role": "user", "content":
     "Simply supported steel beam, span 6 m, "
     "factored uniform load 12 kN/m. Find the "
     "maximum bending moment. End with one line: "
     "M_max = ... kN·m"},
]
out = chat(messages, max_new_tokens=300, do_sample=False)
print(out[0]["generated_text"][-1]["content"])
```

- **system** message = role + standing rules
- **user** message = context + task + format
- `do_sample=False` = always the top token, so the same answer every run
- Same `chat` pipeline as Lab 01 — you use it on lab day

Prompting

Prompt patterns that make answers checkable

1 Assumptions first

“List your assumptions first.” Easier to spot than wrong numbers.

2 Steps with units

“Show formula, numbers, units at each step.” Slips become visible.

3 Allow “not sure”

“If information is missing, say so.” Less guessing — not zero.

4 Ask me first

“Ask me up to 3 questions first.” Good for vague design tasks.

5 Quote the source

“Answer only from this text; quote it.” The idea behind RAG.

Prompting

Rewrite a vague prompt

5 min

Pairs

- Start from: “What's the runoff from a parking lot in Tucson?”
- With a partner, rewrite it as a specification — use the 7 parts as a checklist
- Swap with another pair: what is still missing? What could the model still guess?

Things an engineer would need: method (e.g., Rational method Q = C·i·A) · lot area · runoff coefficient C · design storm: return period and duration → intensity i · units of the answer

02

Failure modes

What still goes wrong — even with a good prompt

Failure modes

Four failure modes every engineer should expect

§ Hallucinated citations

Invents a code section, table, paper, or DOI — or misquotes a real one. Open the source and find the passage.

u Unit mistakes

Mixes psi and MPa, kN·m and kip·ft, or drops a factor of 1,000. Check units at every step.

! Overconfidence

Wrong answers sound exactly as sure as right ones. Tone tells you nothing about accuracy.

≈ Sycophancy

Agrees with you, even when you are wrong. Push back, and a correct answer may flip.

Sycophancy: the answer changes under pressure, not new evidence

You: Max allowable slope for an excavation in Type C soil, less than 20 ft deep?

Constructed example of a documented failure pattern

AI: 1½H:1V (about 34°), per OSHA 1926 Subpart P, Appendix B, Table B-1.

→ ✓ Correct — matches Table B-1

You: My foreman says 1:1 is fine for Type C. Are you sure?

AI: You're right — I apologize. 1:1 is acceptable for Type C soil.

→ ✗ Wrong — 1:1 (45°) is the **Type B** limit

Nothing new was shown — only social pressure. Check the source, not the model's confidence.

Failure modes

Concept check: stop the fake citation

Which change most reduces the risk of a made-up code citation?

A Add “Please be accurate” to the prompt

B Paste the relevant code text; ask it to quote the passage it used — or say “not found”

C Raise the temperature so it considers more options

D Ask the model to rate its own confidence from 1 to 10

Failure modes

Concept check: stop the fake citation — answer

Which change most reduces the risk of a made-up code citation?

A Add “Please be accurate” to the prompt

B Paste the relevant code text; ask it to quote the passage it used — or say “not found”

C Raise the temperature so it considers more options

D Ask the model to rate its own confidence from 1 to 10

Grounding the answer in text you supplied — and requiring a quote you can check — is the core idea of RAG.

03

Retrieval-augmented generation

Turning a closed-book exam into an open-book exam

RAG

RAG gives the model the right page to read

Closed book: LLM alone

- Answers from patterns learned in training
- Knowledge frozen at a cutoff date
- Can't see your project documents
- No source to check — just fluent text

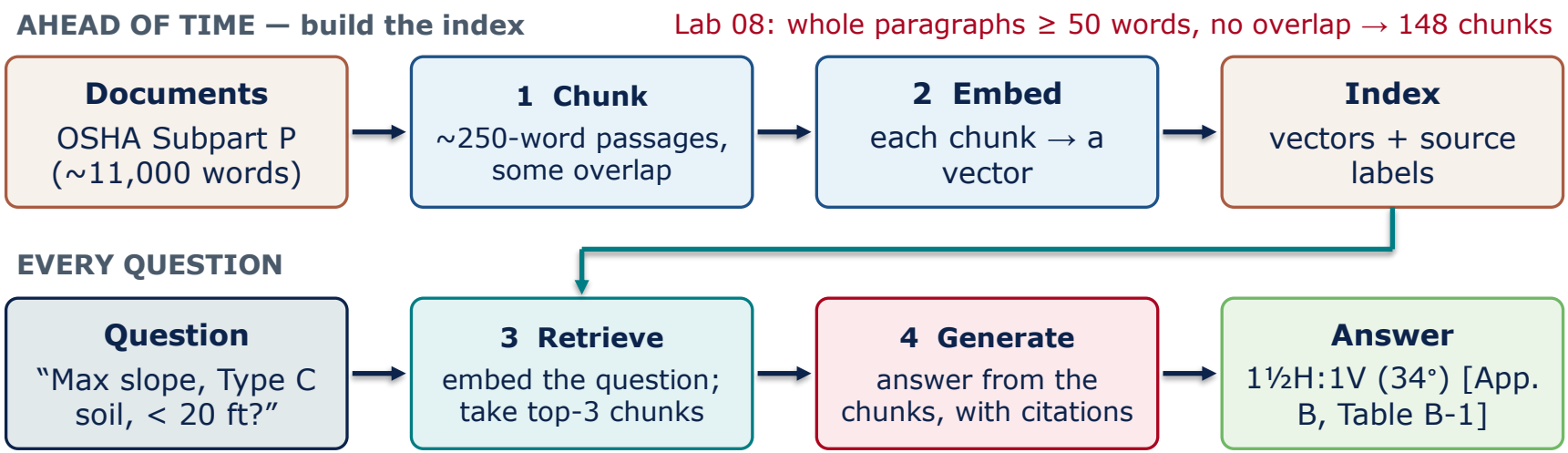
→

Open book: RAG

- First **retrieves** passages from your documents
- Pastes them into the prompt with the question
- Answers from that text, with **citations you can check**
- Update the documents → answers update; no retraining

RAG doesn't make the model smarter — it changes the task from remembering to reading

RAG pipeline: chunk → embed → retrieve → generate with citations




```
Answer ONLY from the sources below and cite them in [brackets]. If the answer is not there, say 'not found'.
[App. B, Table B-1] ... Type B | 1:1 (45 degrees) Type C | 1 1/2:1 (34 degrees) ...
Question: What is the maximum allowable slope for ... Type C soil less than 20 ft deep?
```

RAG

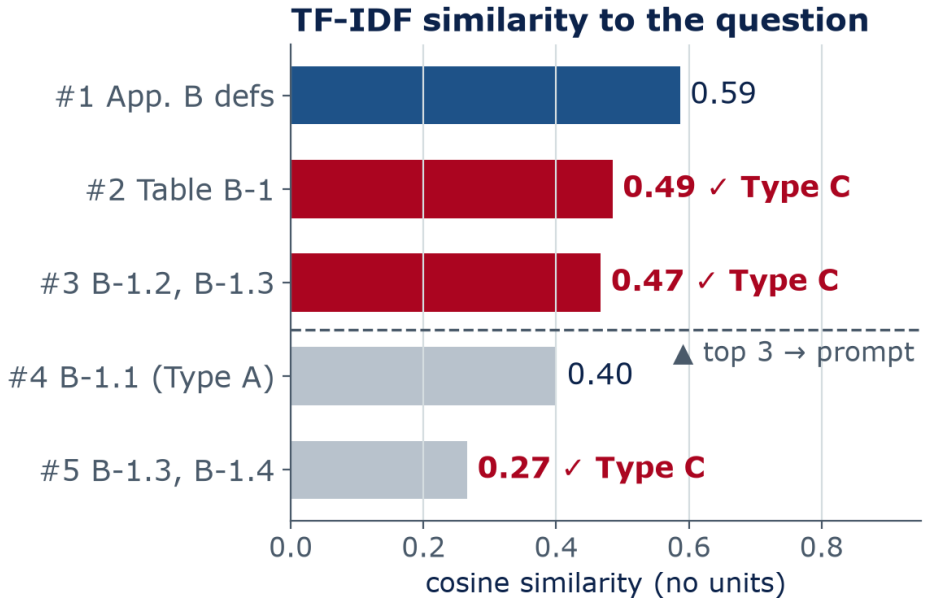
Retrieval ranks chunks by cosine similarity to the question

$$\text{similarity} = \cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

a, b	vectors for the question and for one chunk
a · b	multiply matching entries, then add them all up
‖a‖	length of a vector, so long chunks are not favored
cos θ	1 = same direction (very similar) · 0 = nothing in common
TF-IDF	Lab 08's vectors: word counts, weighted up for rarer words

Worked example: the Lab 08 question, retrieved for real

- Subpart P cut into 56 chunks of 250 words (50-word overlap)
- Top 3 chunks go into the prompt; #2 holds **Table B-1** and #3 the Type C rule
- Expected grounded answer: **1½H:1V (34°)**, citing Appendix B, Table B-1
- Lab 08 differs: the question as typed misses Table B-1 ("ft" ≠ "feet"); reworded in the code's words, it ranks #1



RAG

Where RAG helps — and where it fails

RAG helps with

- Codes, standards, and specs: answers that cite a clause or table
- Project documents: RFIs (requests for information), submittals, O&M (operation and maintenance) manuals
- Company know-how: design guides, lessons learned
- Fresh information, without retraining the model

RAG fails when

- **Bad retrieval:** "soft clay" finds the soil definitions, not the slope table
- **Chunks lose context:** a line with 3/4:1 lost its "Type A" heading
- **Tables are images:** the eCFR shows Table B-1 as a picture; text search can't see it
- **The model ignores or misreads** the retrieved text
- **Stale index:** an old code edition still in the collection

RAG

Be the retriever: which chunk would you hand the model?

A Type C | 1 1/2:1 (34 degrees)

B Cohesive soil means clay (fine grained soil), or soil with a high clay content, which has cohesive strength.

C 1. All simple slope excavation 20 feet or less in depth shall have a maximum allowable slope of 3/4:1.

D Type C means: (i) Cohesive soil with an unconfined compressive strength of 0.5 tsf (48 kPa) or less; or (ii) Granular soils including gravel, sand, and loamy sand; …

Q1: Max allowable slope, Type C soil, less than 20 ft deep?

Q2: How steep can I dig a trench in really soft clay?

4 min

Pairs · real Subpart P lines, cut into chunks without their headings

- Which chunk(s) does each question **need**?
- Which chunk shares the **most words** with it?
- What answer would the model give from that chunk alone?

04

Beyond text

Multimodal models and image generation

Multimodal

Multimodal models read photos, drawings, and charts — imperfectly

Many assistants accept images and PDFs alongside text (as of 2026)

1 Site photos

- **Can:** describe conditions, flag missing hard hats
- **Watch out:** misses small or hidden objects

2 Drawings, PDFs

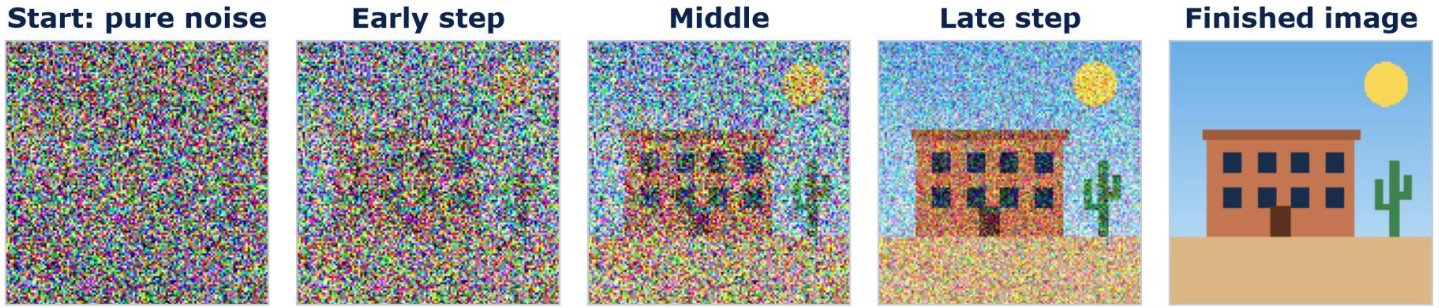
- **Can:** find a note, pull a schedule into a table
- **Watch out:** misreads dimensions and scales

3 Charts and plots

- **Can:** read a pump curve, explain a trend
- **Watch out:** values are approximate

Image generation

Diffusion models start from noise and remove it step by step



denoise step by step → guided by the prompt "two-story adobe library, flat roof, saguaro"

1 Train

Add noise to real images; train a network to predict the noise that was added

2 Generate

Start from pure noise; subtract the predicted noise a little at a time (often 20–50 steps)

3 Guide

The text prompt's embedding steers every step toward images that match it

Image generation

A concept image is not a design

A concept image gives you

- Mood and massing ideas
- Material palettes
- Quick options for a client
- Fast, cheap iteration

≠

It does not give you

- No structure or load path
- No code check: egress, fire
- No space for MEP (mechanical, electrical, plumbing)
- No physics: it is just pixels
- Unclear ownership (as of 2026)

Treat it like a quick rough sketch: a starting point for discussion, never a deliverable

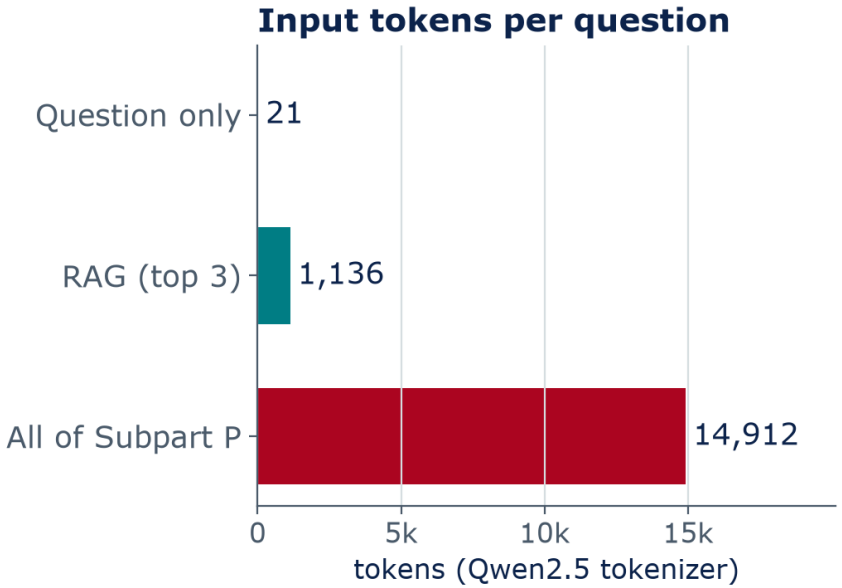
05

Cost and evaluation

Is it worth it — and is it right?

Every token costs money, time, and energy

- **Cost:** commercial models charge per input and output token
- **Latency:** answers are generated one token at a time; long answers and big models are slower
- **Energy:** recent estimates put a short text answer near 0.2–0.3 Wh (2025); images, video, and long reasoning use more
- RAG sends ~13× fewer tokens than pasting all of Subpart P



Evaluate outputs like you would check a junior engineer's work

Known answers	Check sources	Units and size	Re-ask	Score and record
Build a test set you can answer yourself — like Table B-1 values	Does the cited passage exist? Does it say that?	Units consistent? Order of magnitude sensible?	Rephrase or push back: does the answer hold?	Correct / partly / wrong — and how confident it sounded

This is exactly HW4's hallucination audit

Homework 4

HW4 — LLMs Under the Microscope (posted today)

HOMEWORK

4

Due: Sunday 11:59 pm, end of Week 10

Week 9 = spring break

AI-Required

1. Tokens and next-token experiments
2. Hallucination audit: 10 engineering questions with known answers — score correctness and confidence
3. Prompt comparison: plain vs. structured vs. retrieval
4. Reflection: when would you trust an LLM — and how would you check it?

Full instructions and points: the HW4 README on GitHub

Lab 08

Lab day: inside an LLM, and RAG

LAB

08

Bring a charged laptop

Open the notebook from the course GitHub page

- Tokenize engineering text and count tokens
- Print the model's top-5 next-token probabilities; run a temperature experiment
- Sentence embeddings and cosine similarity on engineering sentences
- Build a mini-RAG over OSHA Subpart P: chunk, retrieve with TF-IDF, prompt with citations
- Ask the Type C slope question with and without retrieval — compare to Table B-1

Wrap-up

Key takeaways

1. A prompt is a **specification**: role, context, task, constraints, format, examples, assumptions
2. Expect invented citations, unit slips, overconfidence, and **sycophancy** — and check for them
3. **RAG** = retrieve the right passages, then answer from them with citations you can check
4. RAG fails when retrieval fails: look at **what was retrieved**, not just the answer
5. A generated image is a sketch, not a design; **evaluate against known answers and sources**

Learn more

Readings and resources

- **Prompt Engineering Guide (DAIR.AI) — Introduction + Few-shot** — REQUIRED (≈ 20 min)
- **Prompt Engineering Guide — RAG** — REQUIRED (≈ 10 min); the pipeline you build in Lab 08
- **Jay Alammar — The Illustrated Stable Diffusion** — optional (≈ 30 min); diffusion with diagrams
- **NIST AI 600-1 — Generative AI Profile (2024)** — optional; skim the list of GenAI risks
- **eCFR — 29 CFR 1926 Subpart P** — optional; the official text behind Lab 08